Algorithm

Lecture #12

Syed Hamed Raza

Heap

Heap

- The number of nodes in a complete binary tree of height h is
- $n = 2^0 + 2^1 + 2^2 + \cdots + 2^h$

$$= \sum_{i=0}^{h} 2^i = 2^{h+1} - 1$$

- h in terms of n is

$h = (\log(n + 1)) - 1 \approx \log n \in \Theta(\log n)$

Heap

Heap

- One of the clever aspects of heaps is that they can be stored in arrays without using any pointers.
- This is due to the left-complete nature of the binary tree.
- We store the tree nodes in level-order traversal.

Heap

Heap

Access to nodes involves simple arithmetic operations:

- left(i): returns 2i, index of left child of node i.
- right(i): returns 2i + 1, the right child.
- parent(i) returns $\lfloor i/2 \rfloor$, the parent of i.

Heap sort Algorithm

Heapsort Algorithm

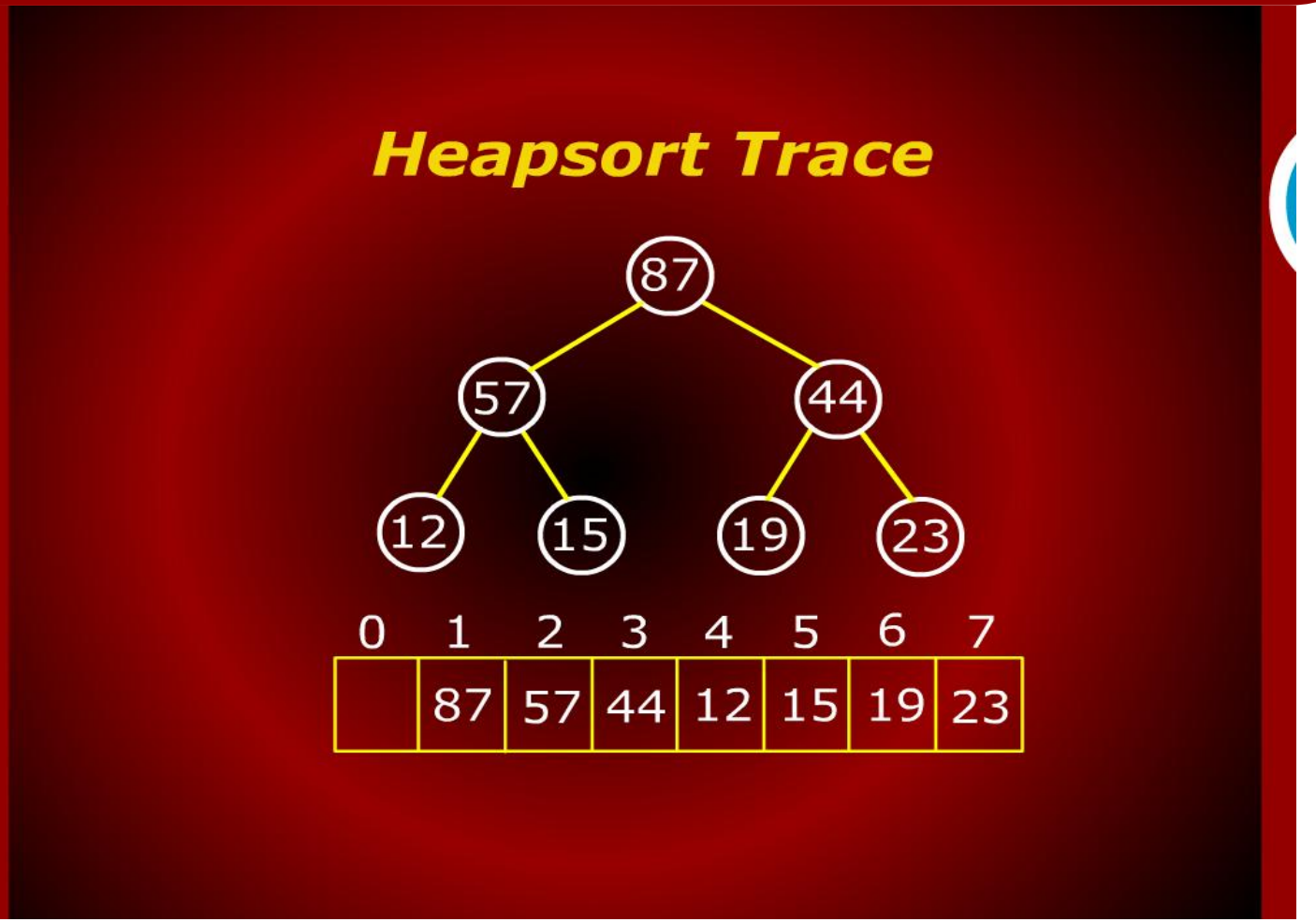
- We build a max heap out of the given array of numbers A[1..n].
- We repeatedly extract the the maximum item from the heap.
- Once the max item is removed, we are left with a hole at the root.
- To fix this, we will replace it with the last leaf in tree.

Heap sort Trace

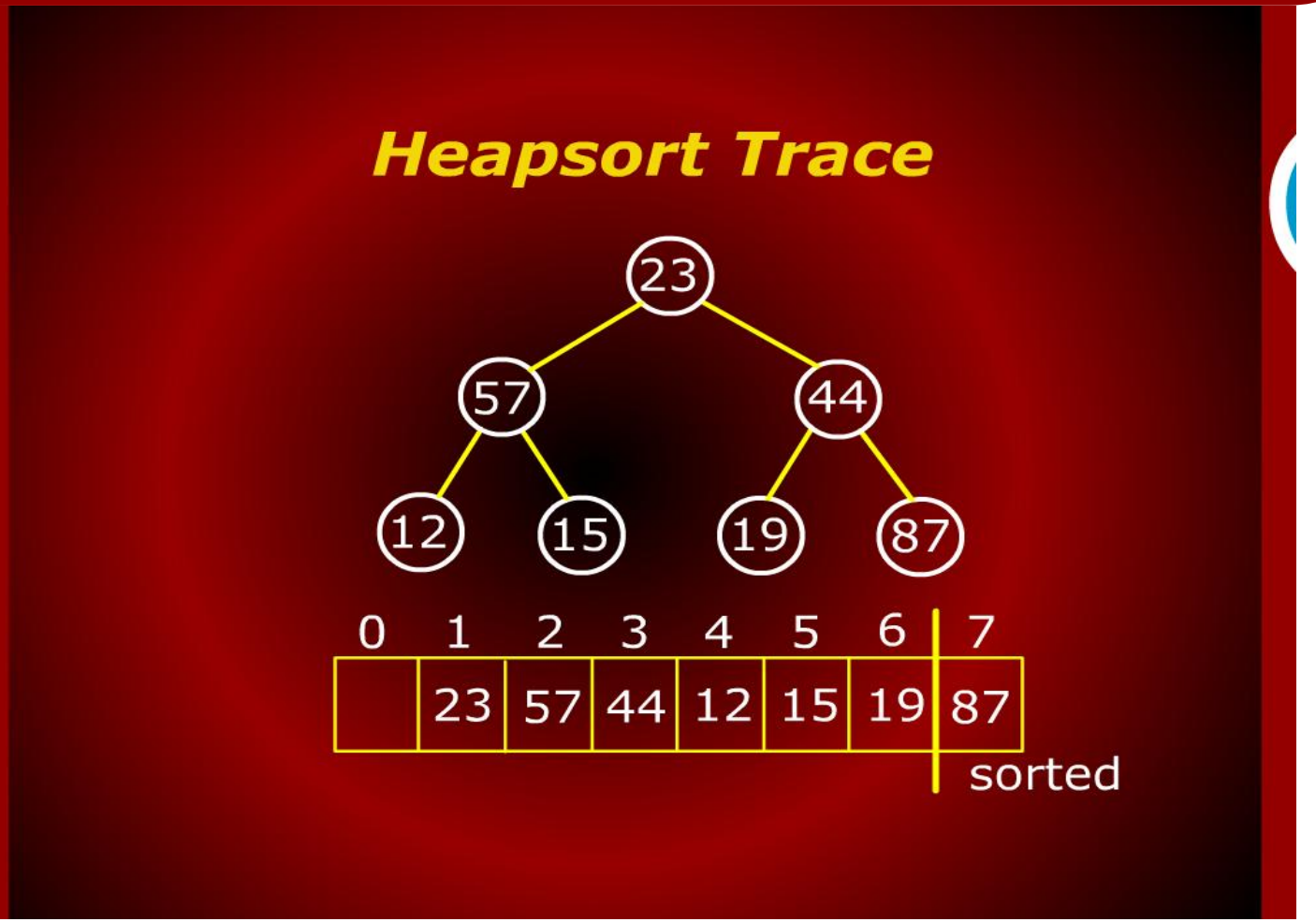
Heap Sort

```
HEAPSORT( array A, int n)
 1 BUILD-HEAP(A, n)
 2 m ← n
 3 while (m ≥ 2)
 4 do SWAP(A[1],A[m])
 5      m ← m – 1
 6      HEAPIFY(A, 1,m)
```

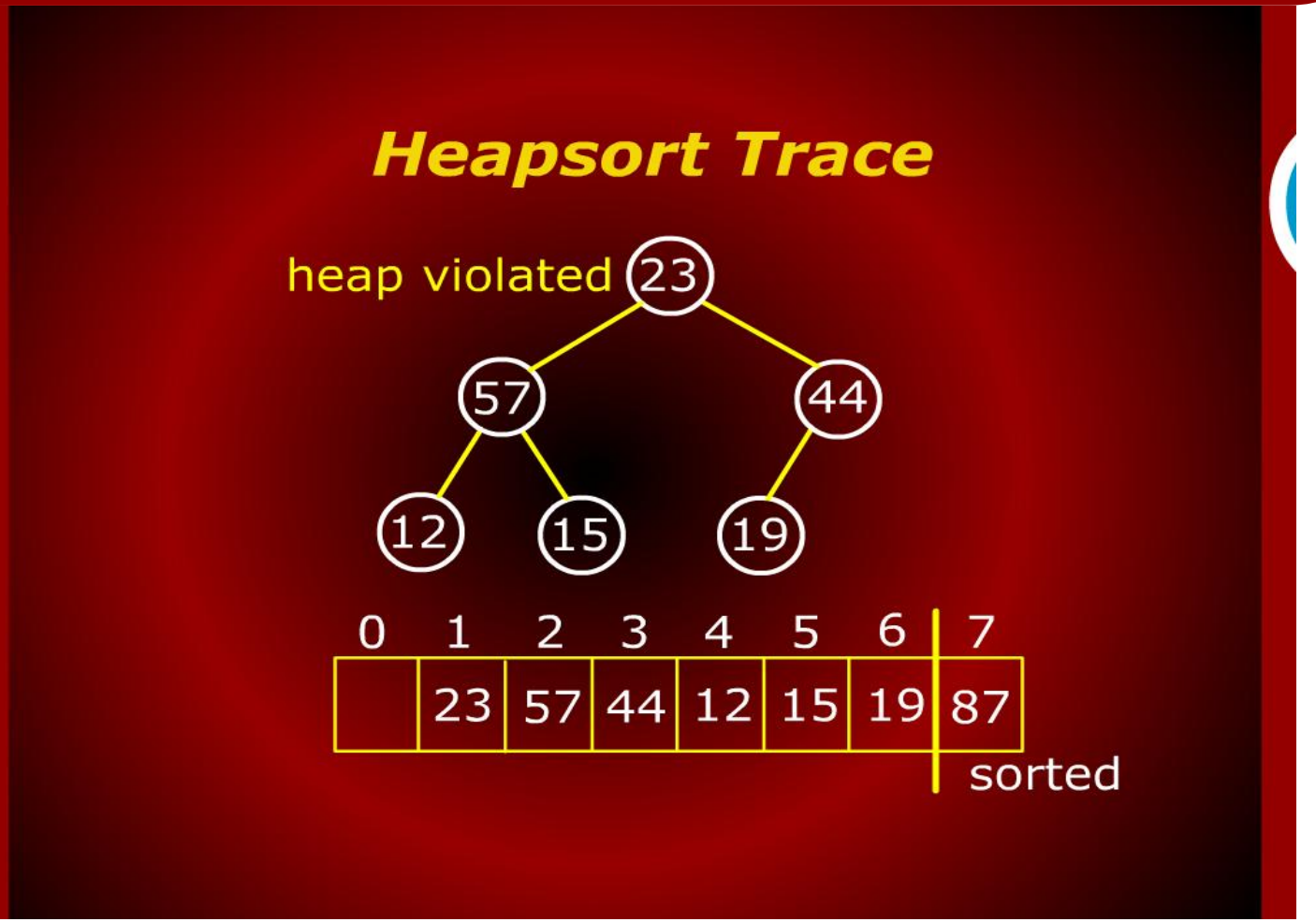
Heap sort Trace



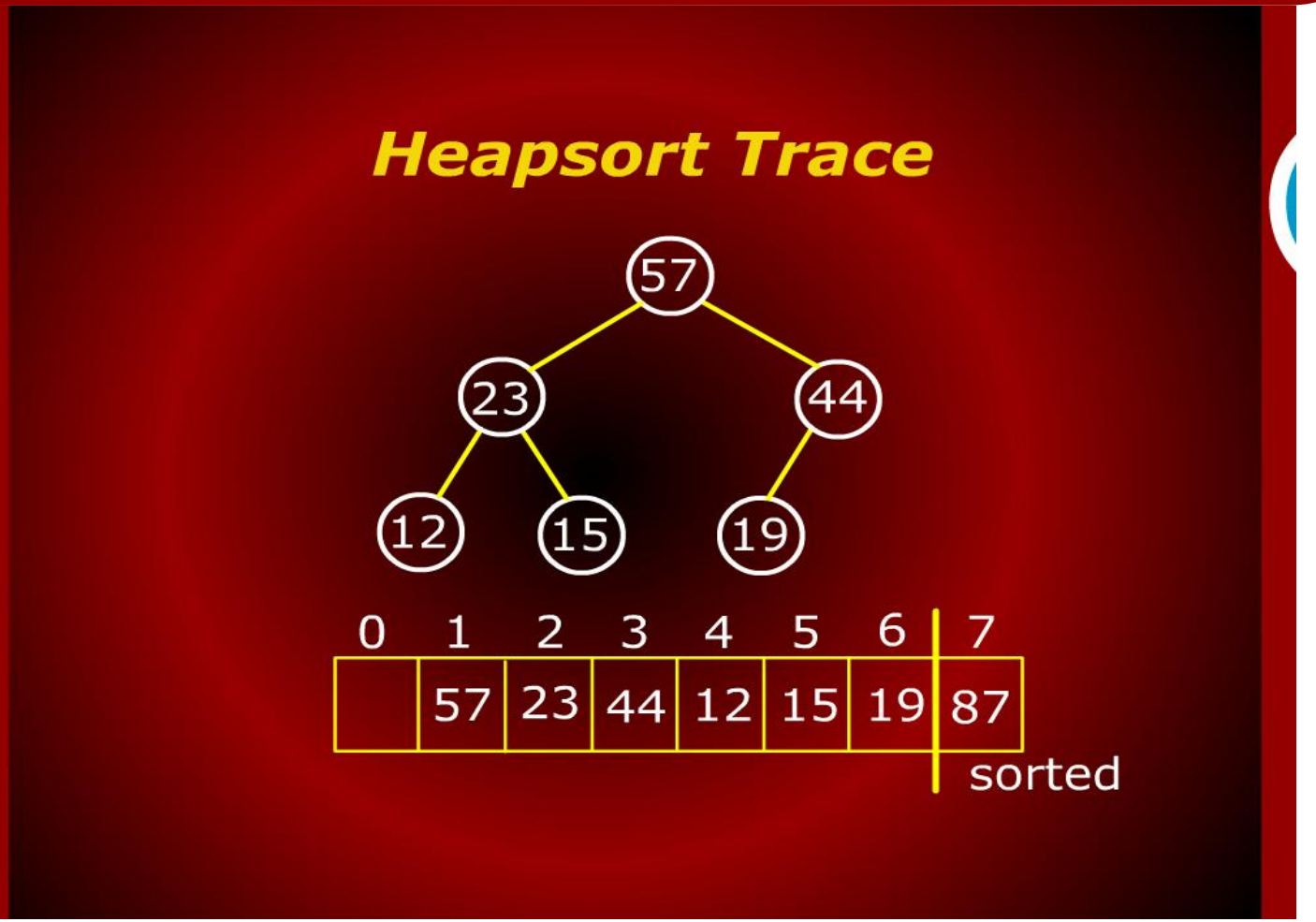
Heap sort Trace



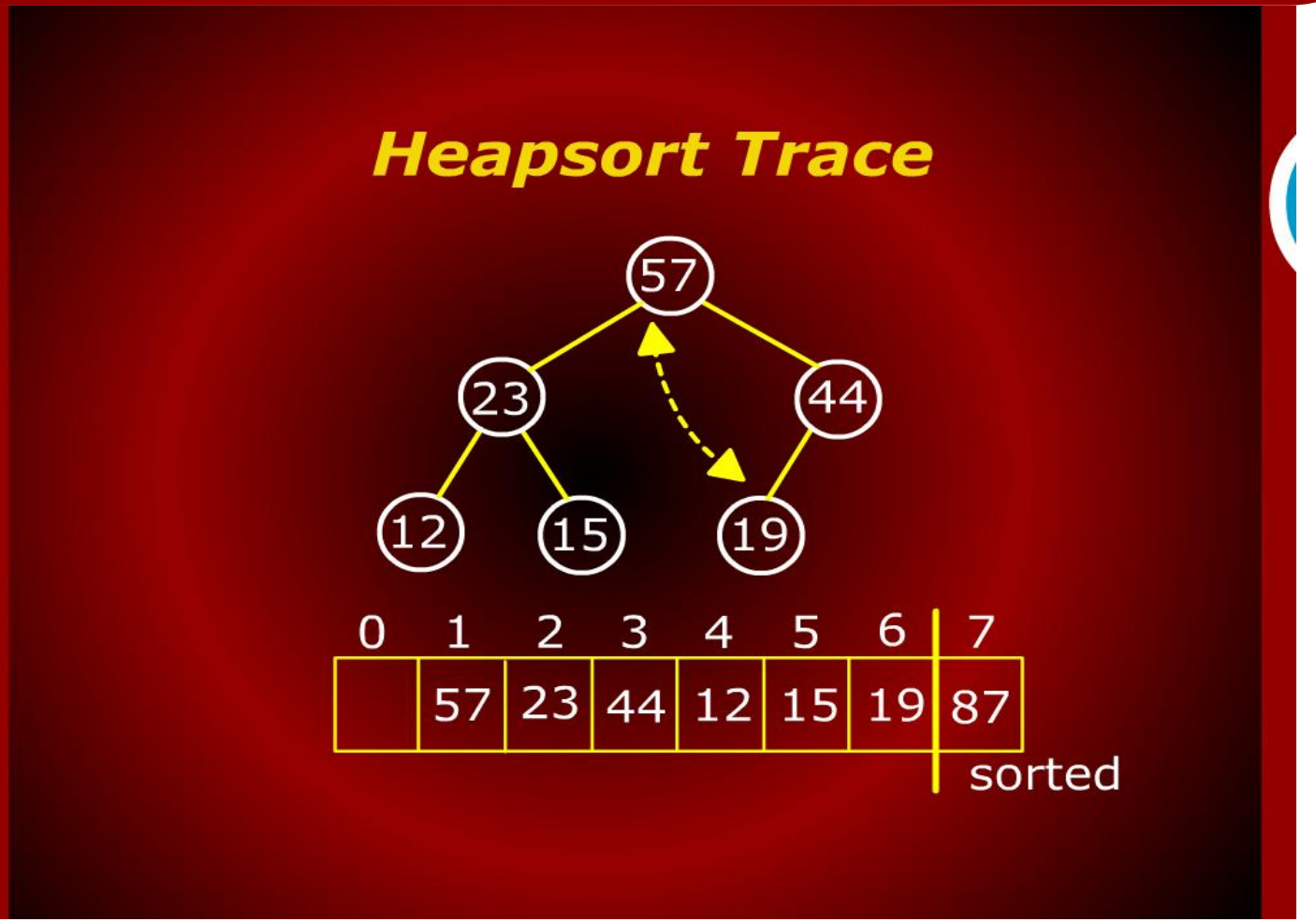
Heap sort Trace



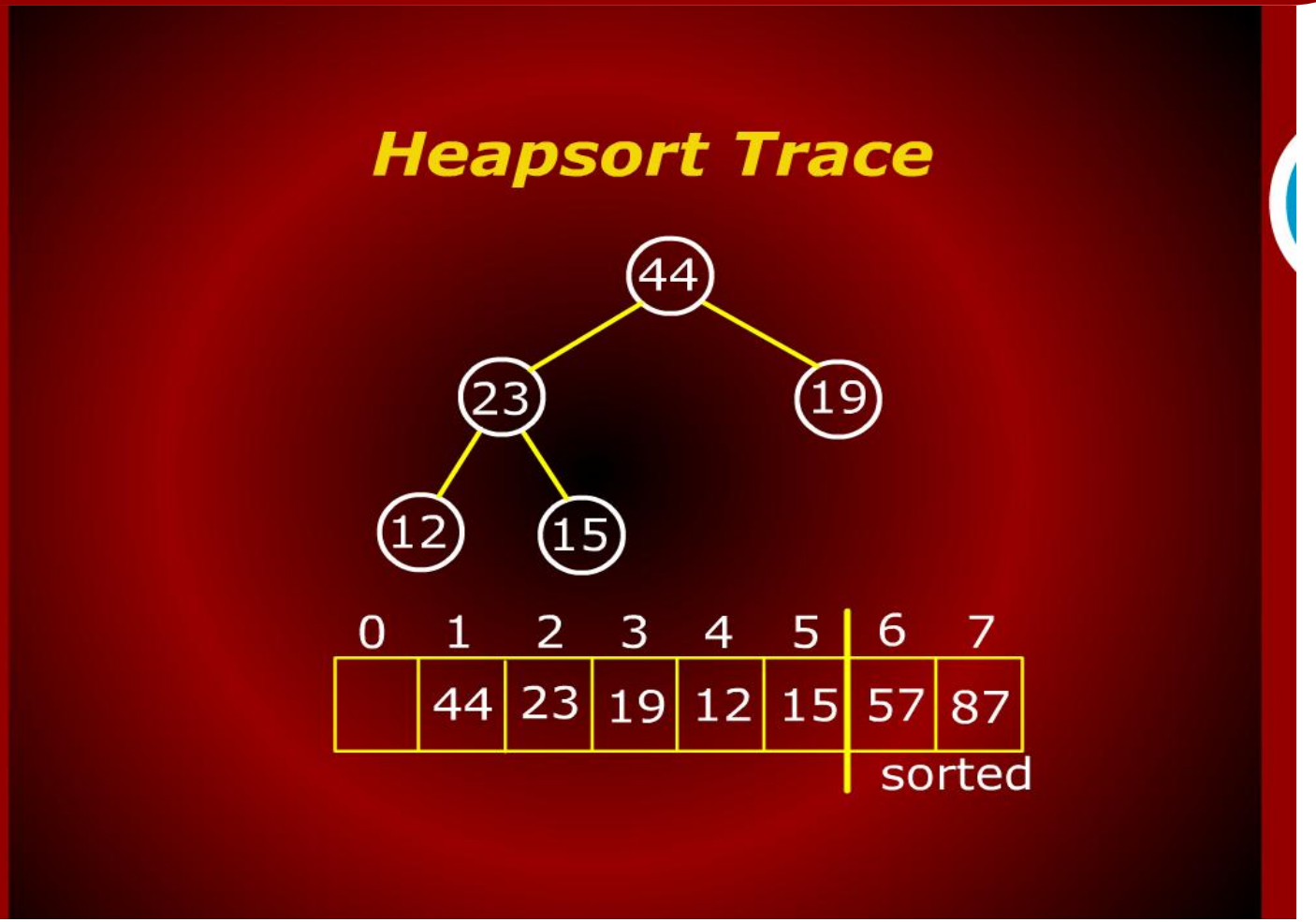
Heap sort Trace



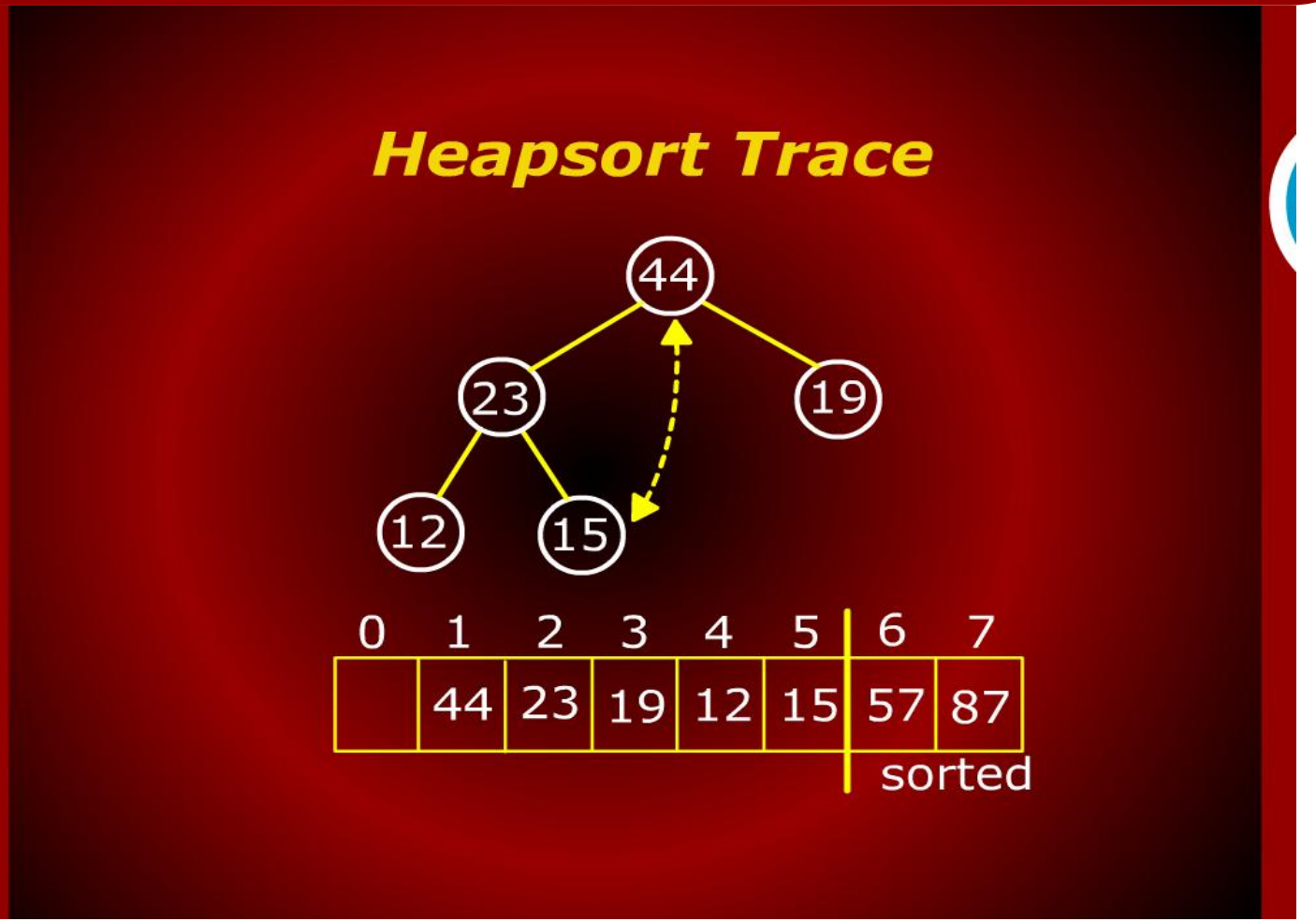
Heap sort Trace



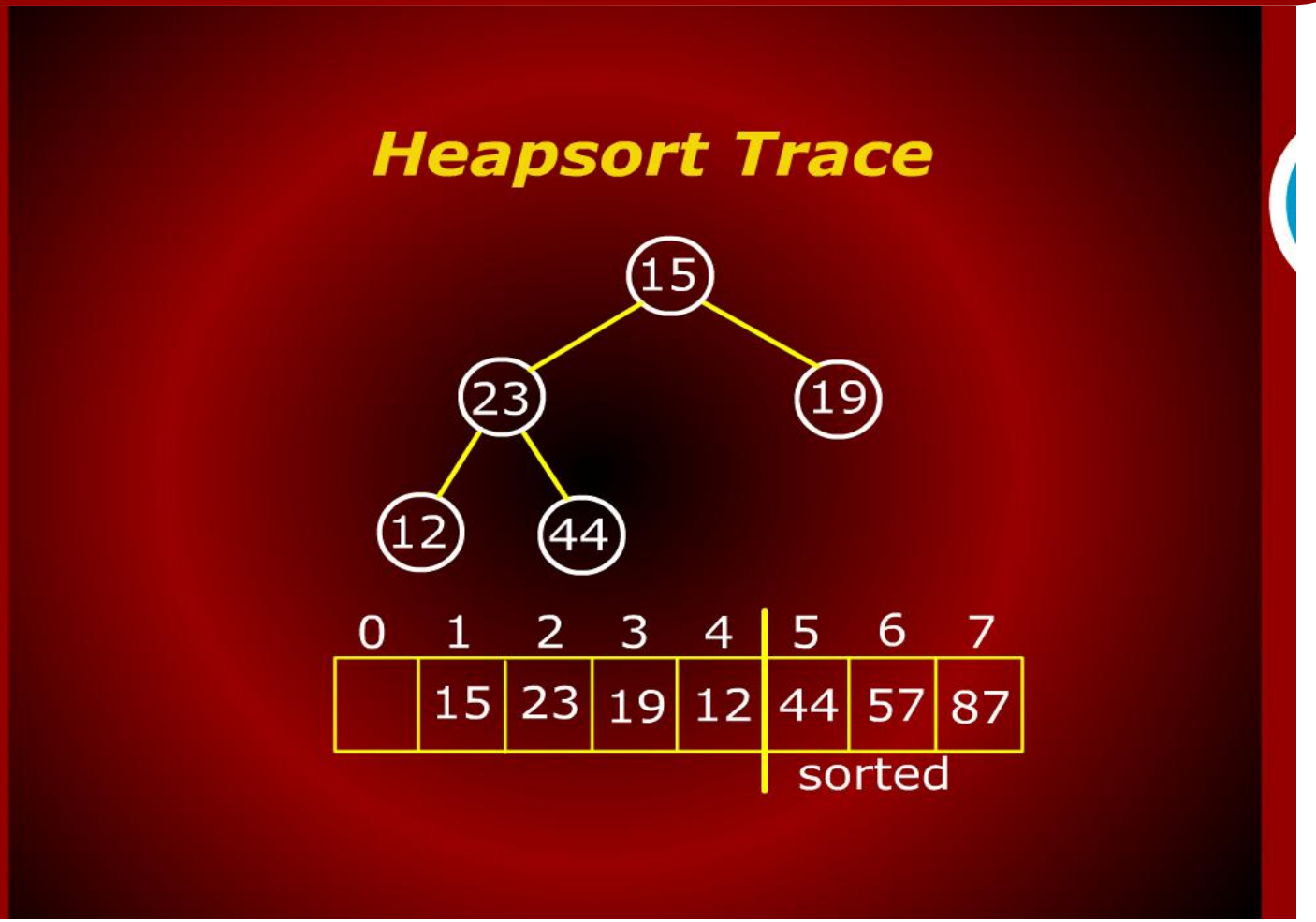
Heap sort Trace



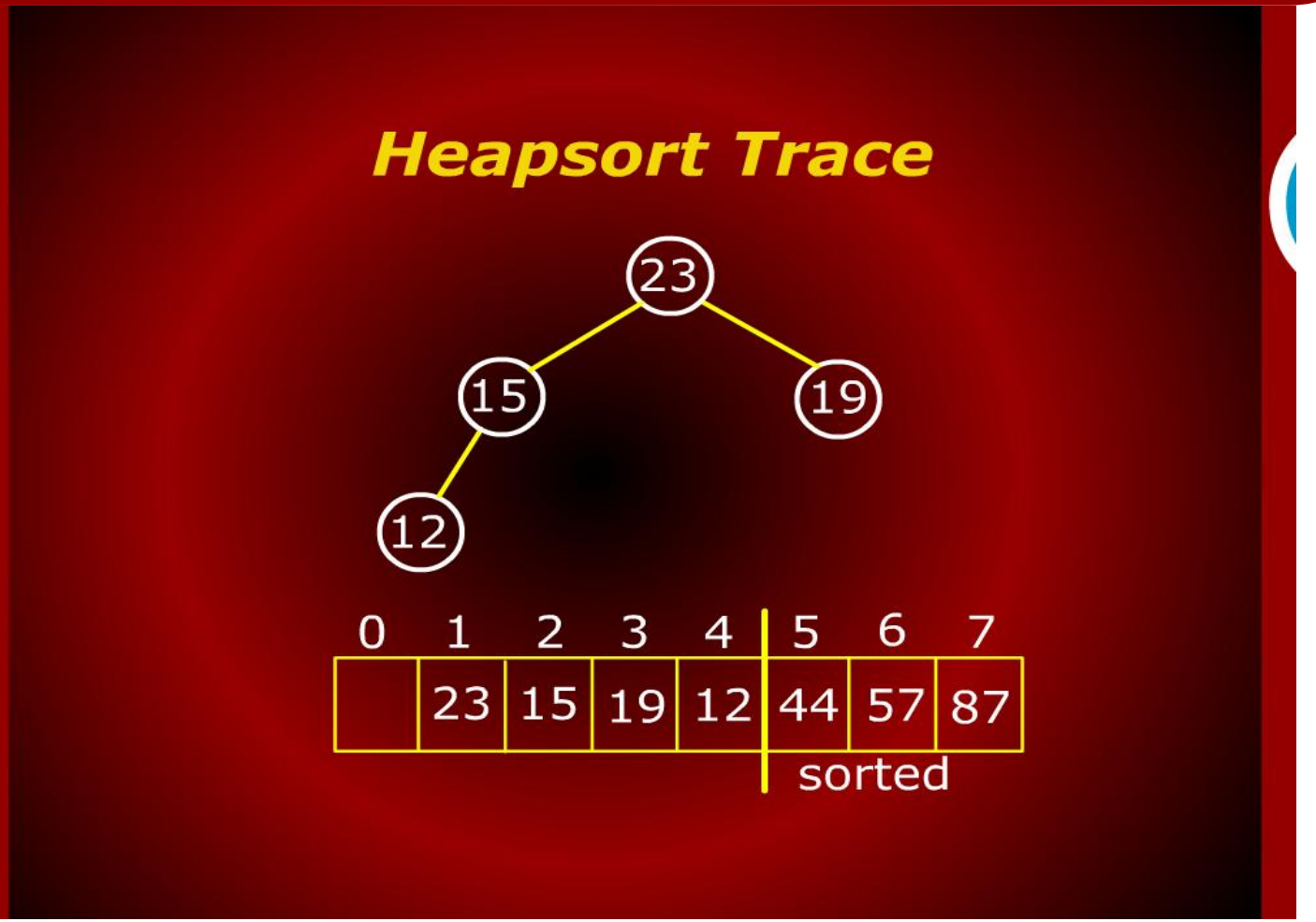
Heap sort Trace



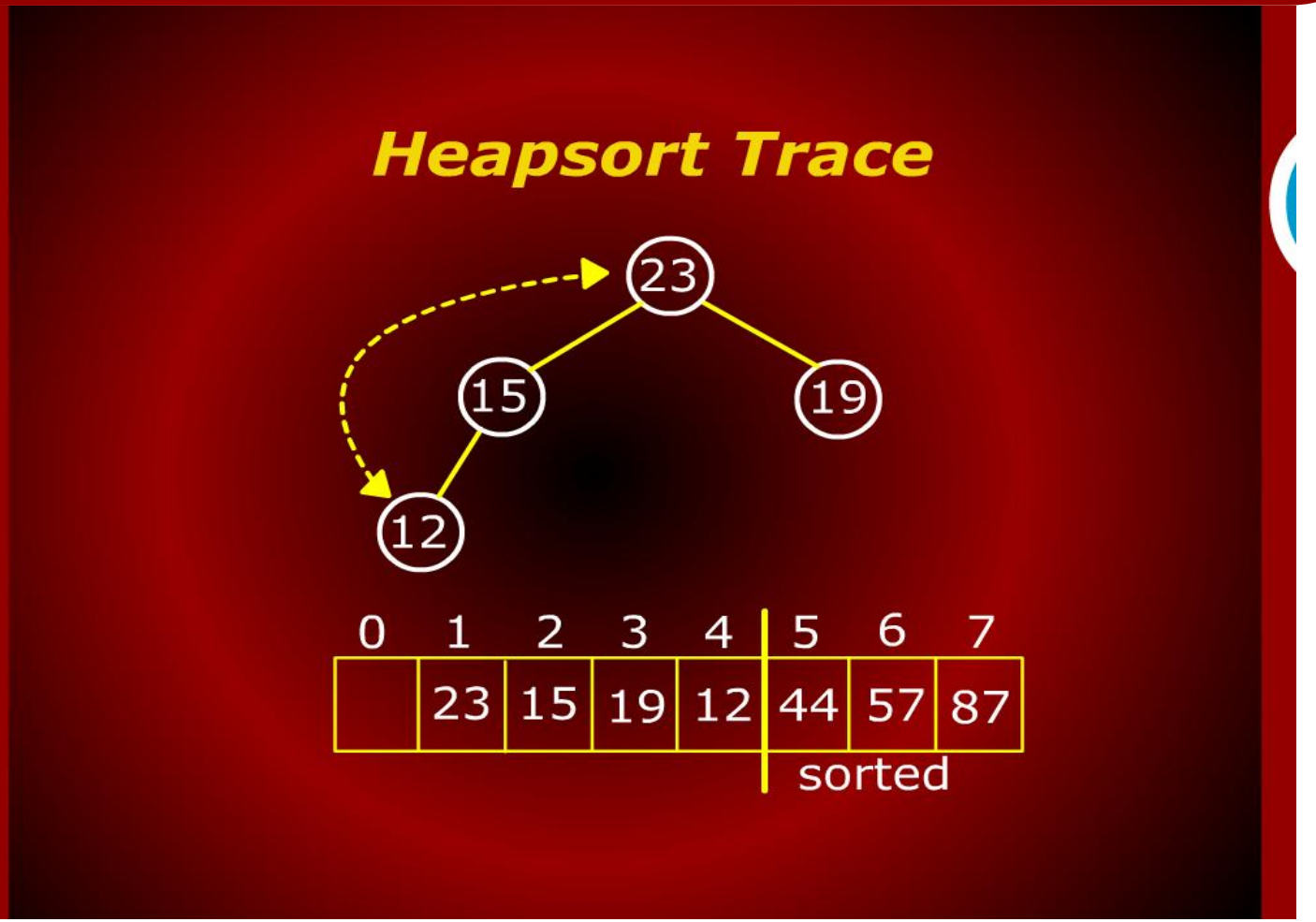
Heap sort Trace



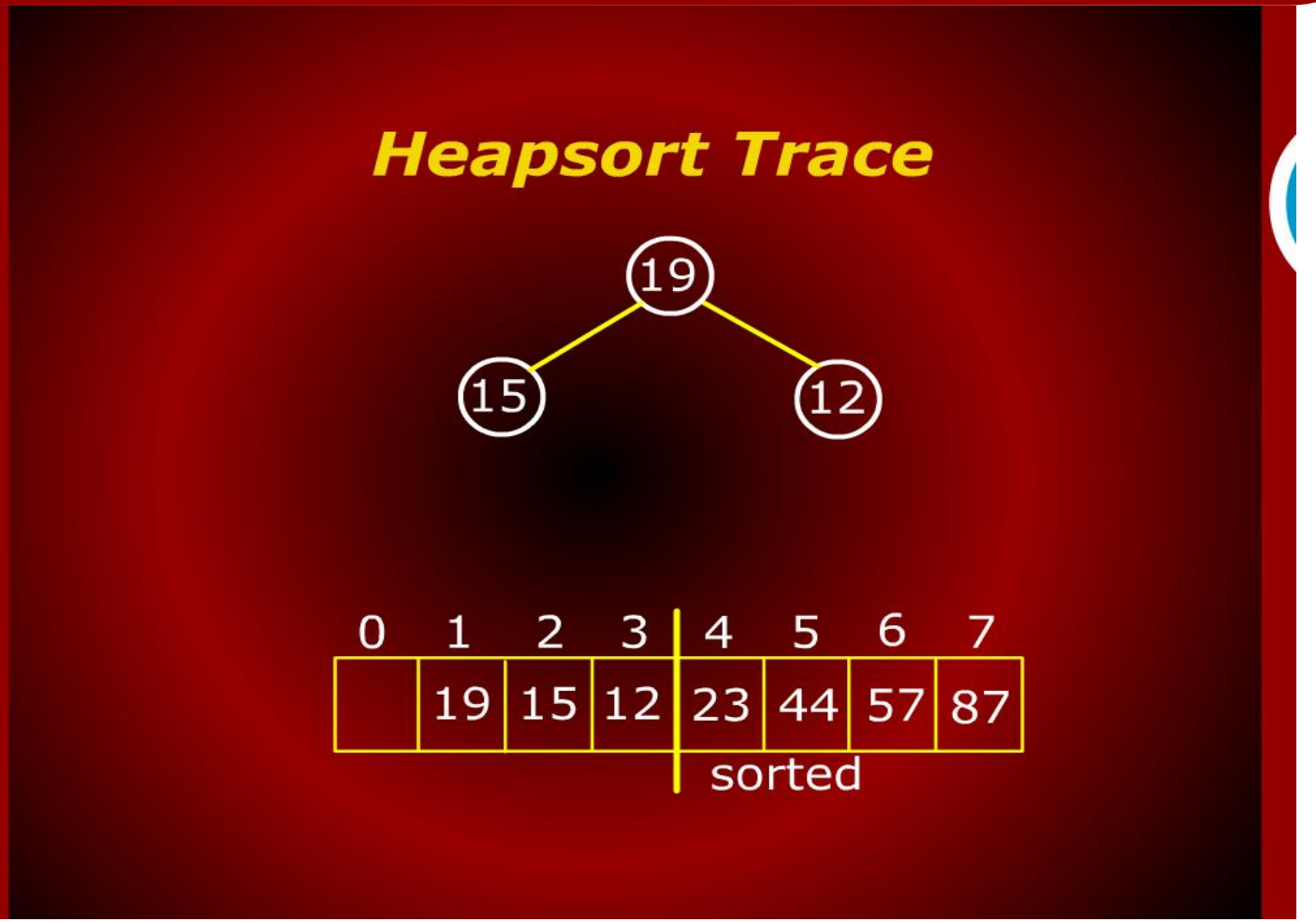
Heap sort Trace



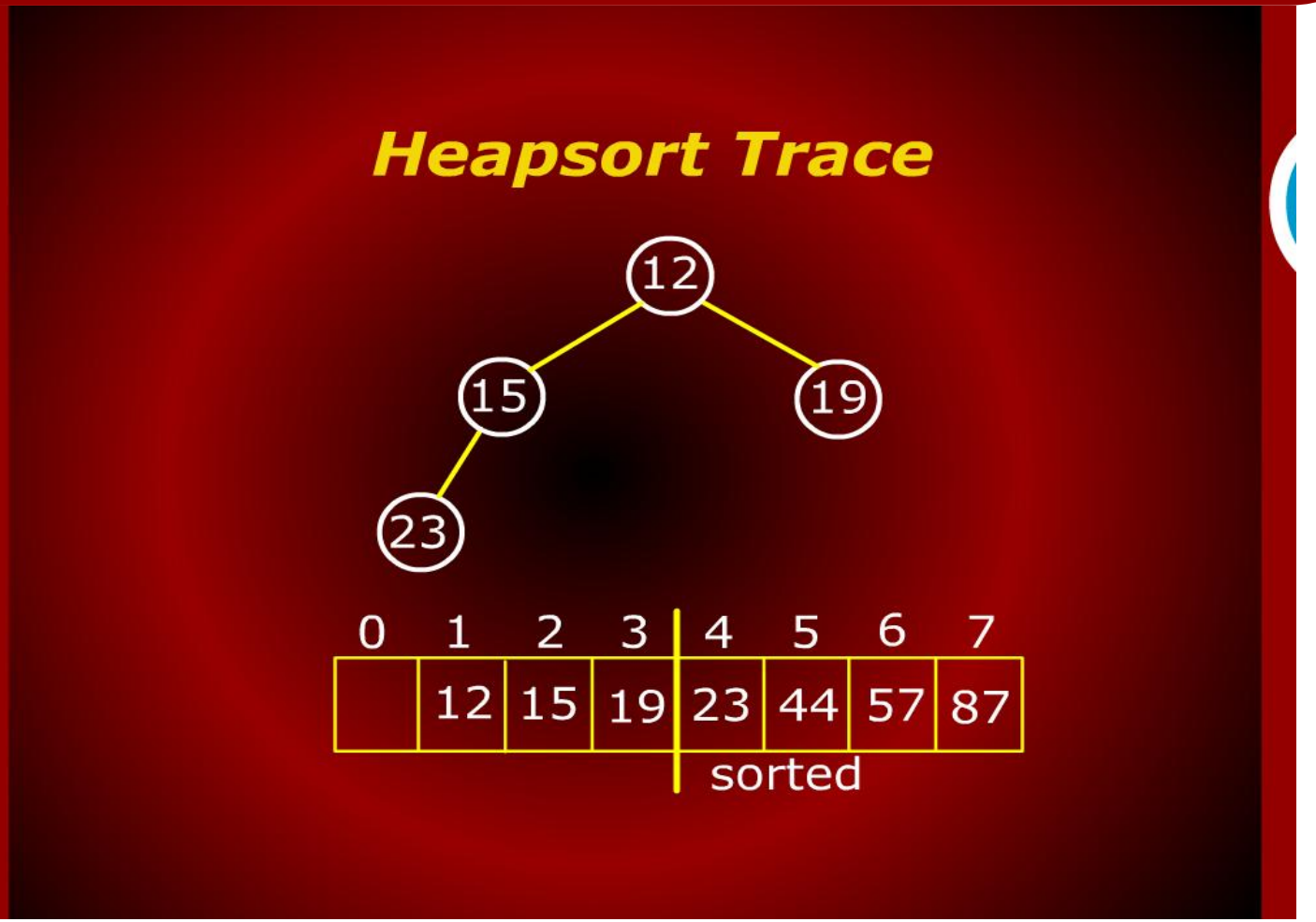
Heap sort Trace



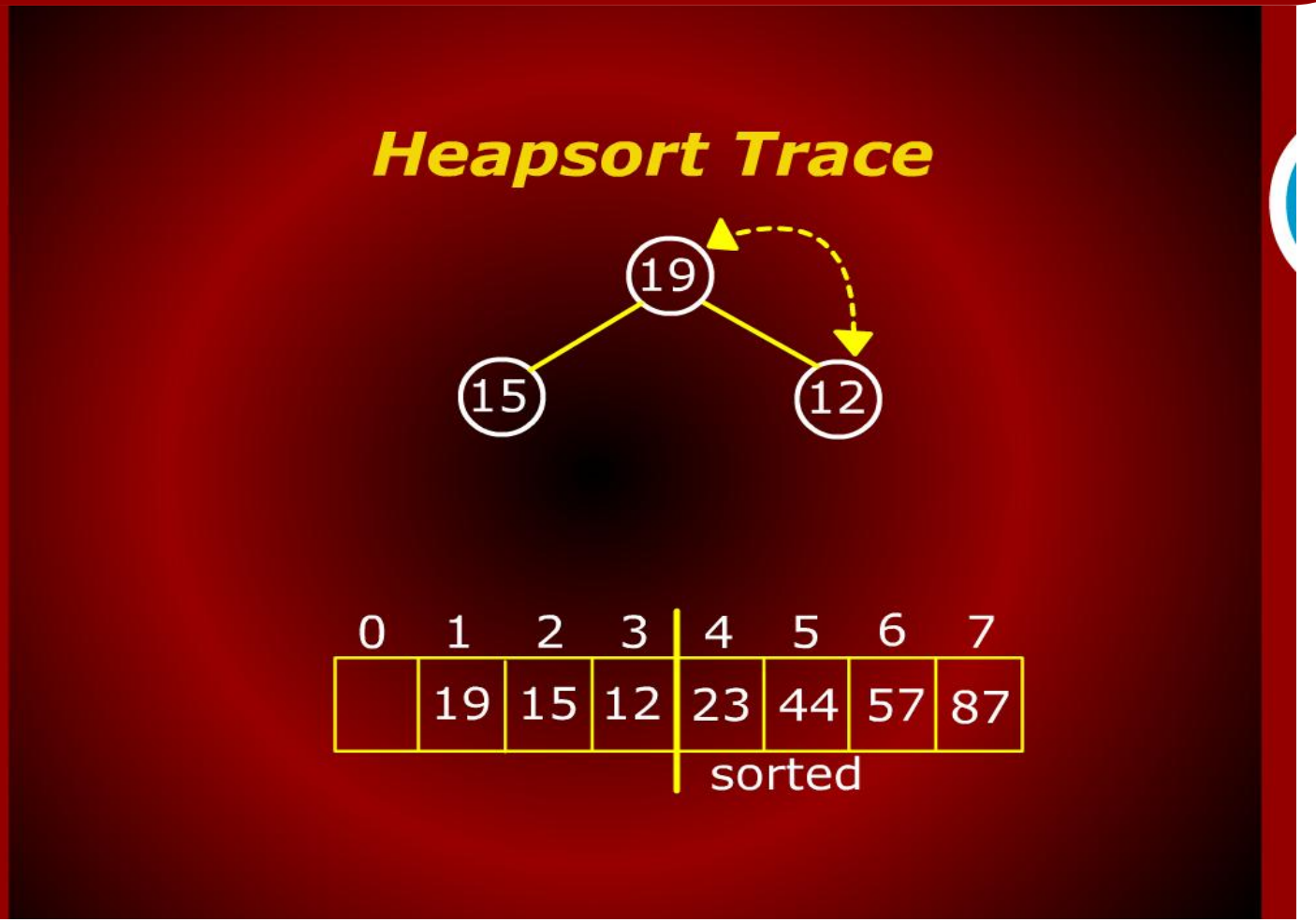
Heap sort Trace



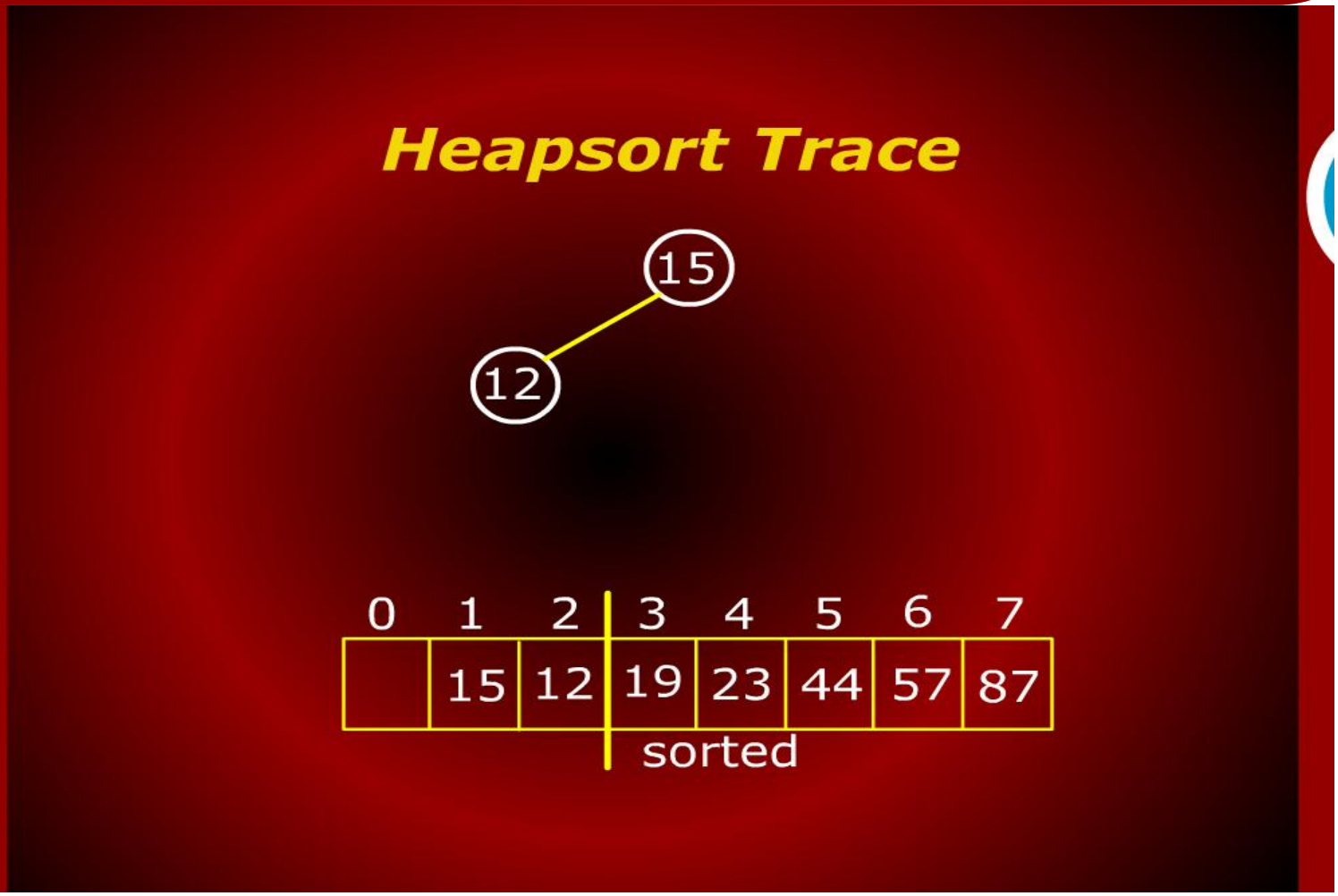
Heap sort Trace



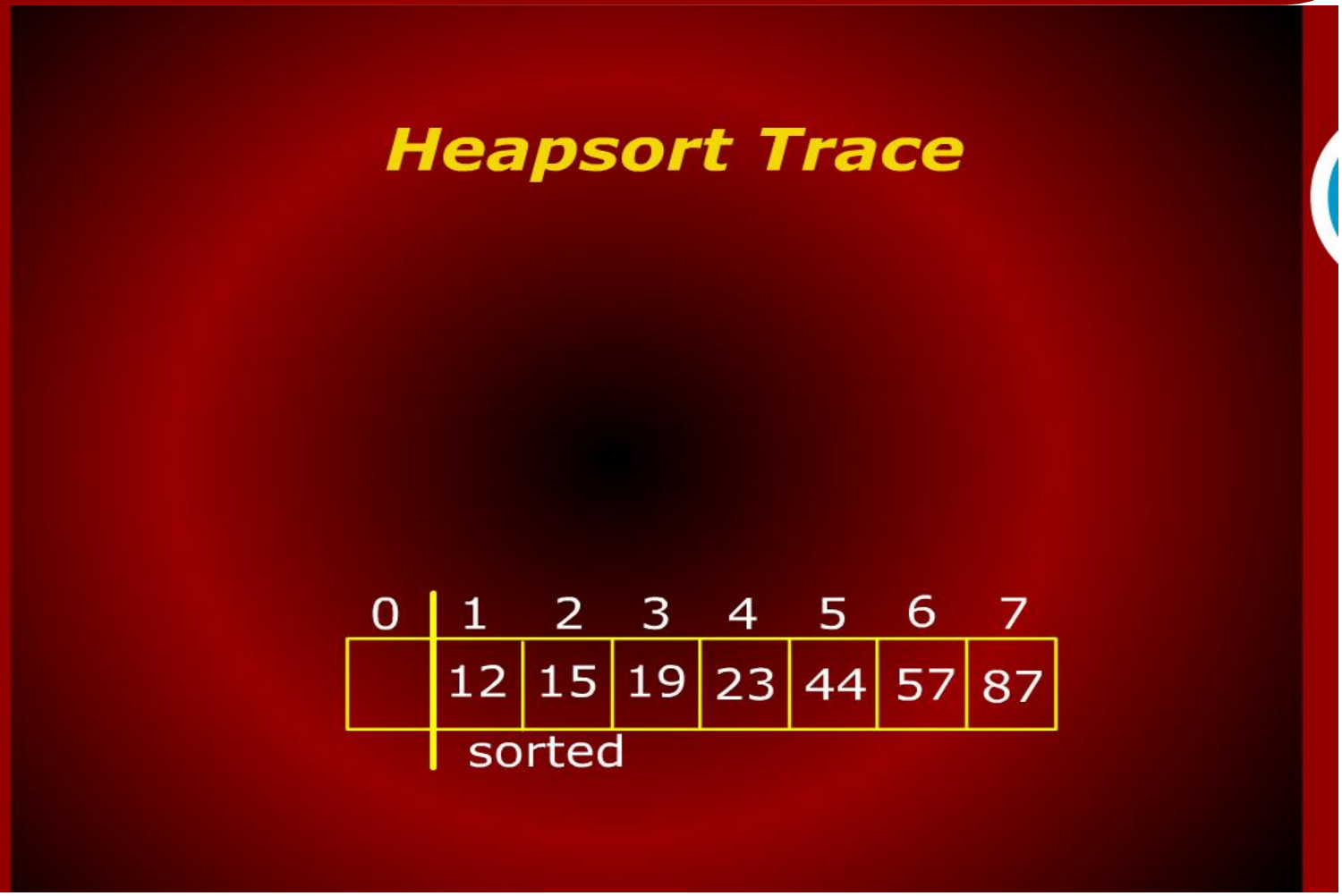
Heap sort Trace



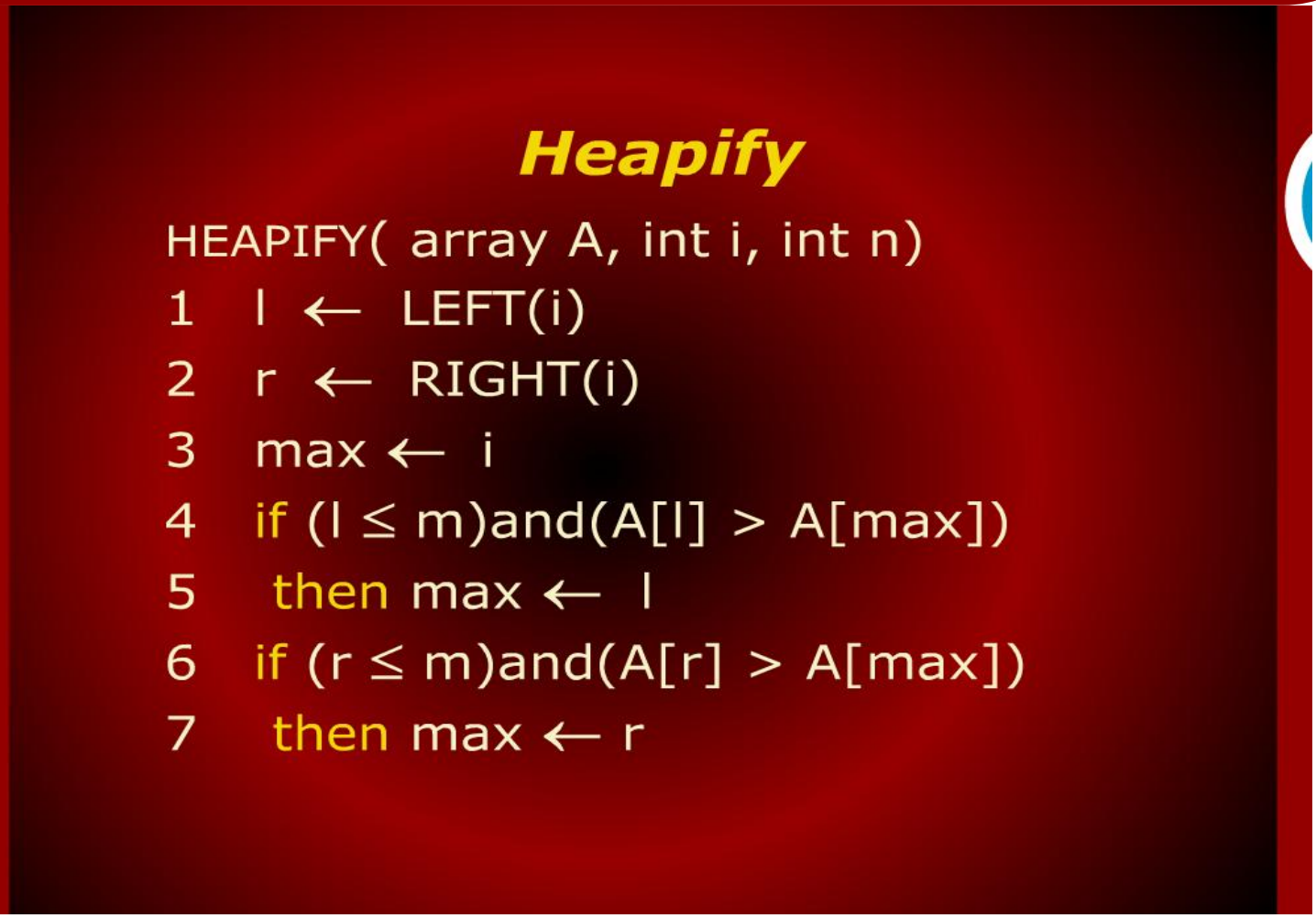
Heap sort Trace



Heap sort Trace



Heapify


Heapify
HEAPIFY(array A, int i, int n)
1 l ← LEFT(i)
2 r ← RIGHT(i)
3 max ← i
4 if (l ≤ m)and(A[l] > A[max])
5 then max ← l
6 if (r ≤ m)and(A[r] > A[max])
7 then max ← r

Heapify

Heapify

```
3   max ← i
4   if (l ≤ m)and(A[l] > A[max])
5    then max ← l
6   if (r ≤ m)and(A[r] > A[max])
7    then max ← r
8   if (max ≠ i)
9   then SWAP(A[i],A[max])
10 HEAPIFY(A,max,m)
```

Analysis of Heapify

Analysis of Heapify

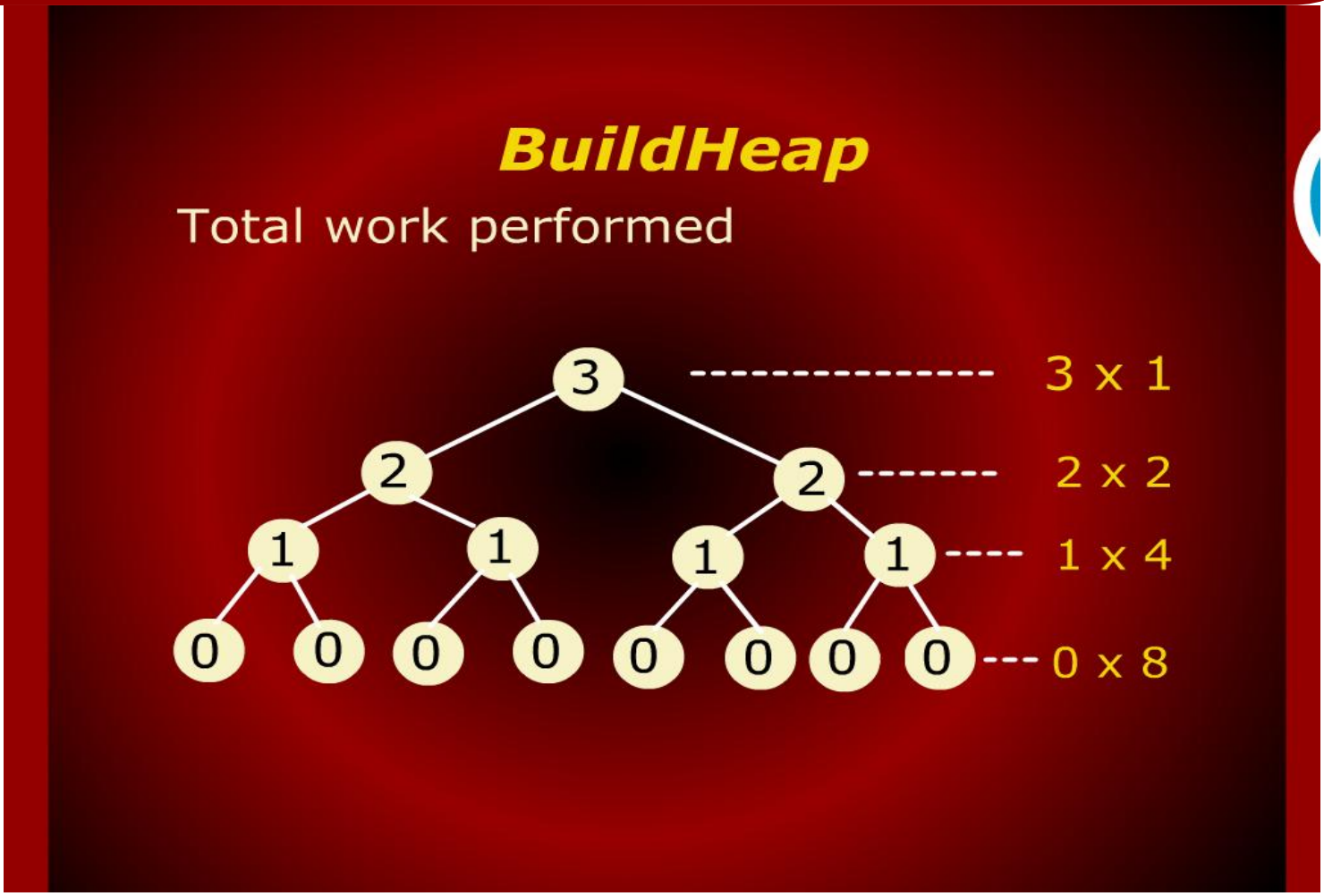
- We call heapify on the root of the tree.
- The maximum levels an element could move up is $\Theta(\log n)$ levels.
- At each level, we do simple comparison which $O(1)$.
- The total time for heapify is thus $O(\log n)$.
- Notice that it is not $\Theta(\log n)$ since, for example, if we call heapify on a leaf, it will terminate in $\Theta(1)$ time.

Analysis of BuildHeap

Analysis of BuildHeap

- For convenience, we will assume $n = 2^{h+1} - 1$ where h is the height of tree..
- The heap is a left-complete binary tree.
- Thus at each level j, $j < h$, there are 2^j nodes in the tree.
- At level h, there will be 2^h or less nodes.
- How much work does buildHeap carry out?

BuildHeap

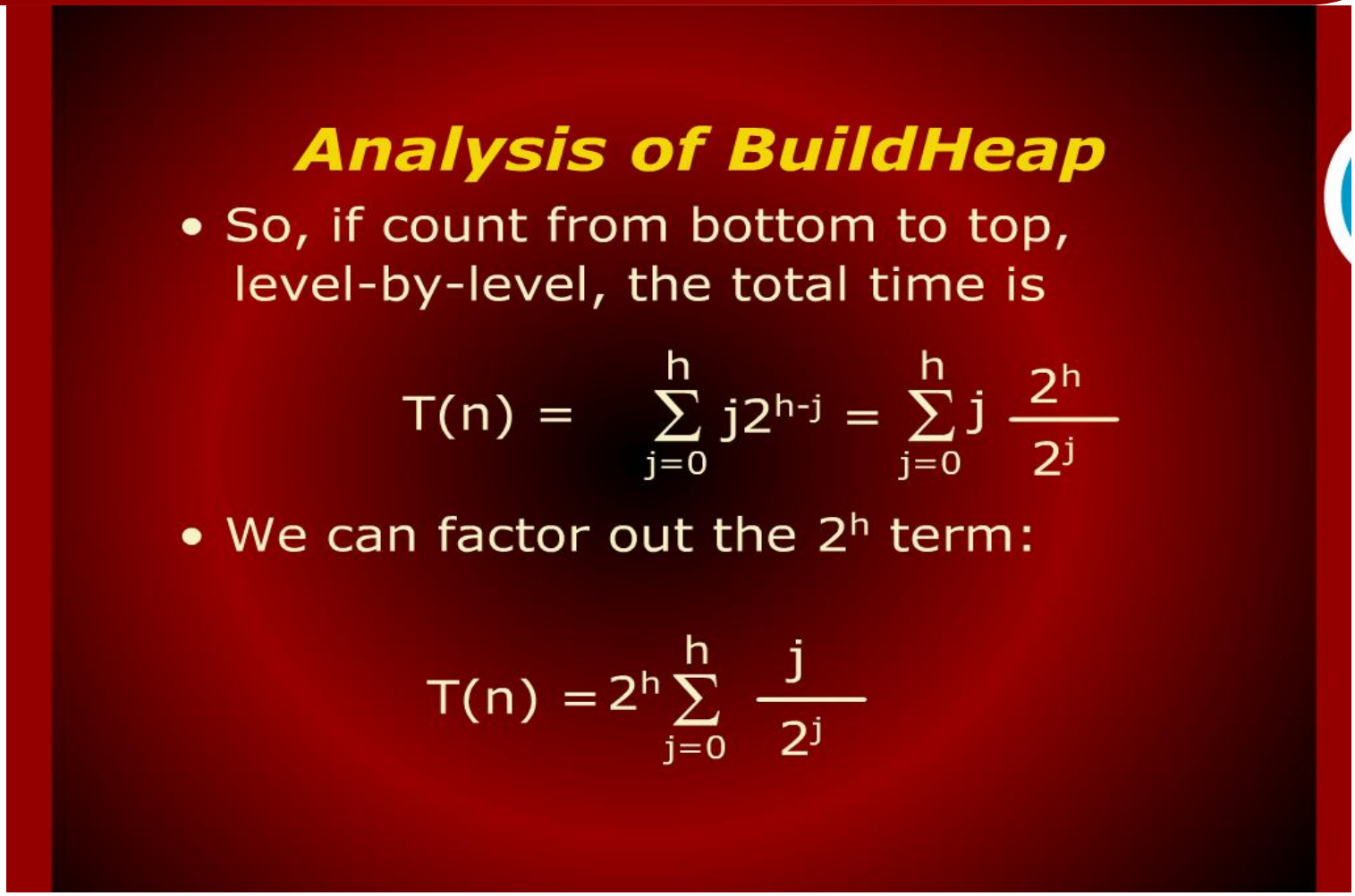

BuildHeap
Total work performed
3
2
2
1
1
1
1
0
0
0
0
0
0
0
0
3 x 1
2 x 2
1 x 4
0 x 8

Analysis of BuildHeap

Analysis of BuildHeap

- At the bottom most level, there are 2^h nodes but we do not heapify these.
- At the next level up, there are 2^{h-1} nodes and each might shift down 1.
- In general, at level j, there are 2^{h-j} nodes and each may shift down j levels.

Analysis of BuildHeap



Analysis of BuildHeap

Analysis of BuildHeap

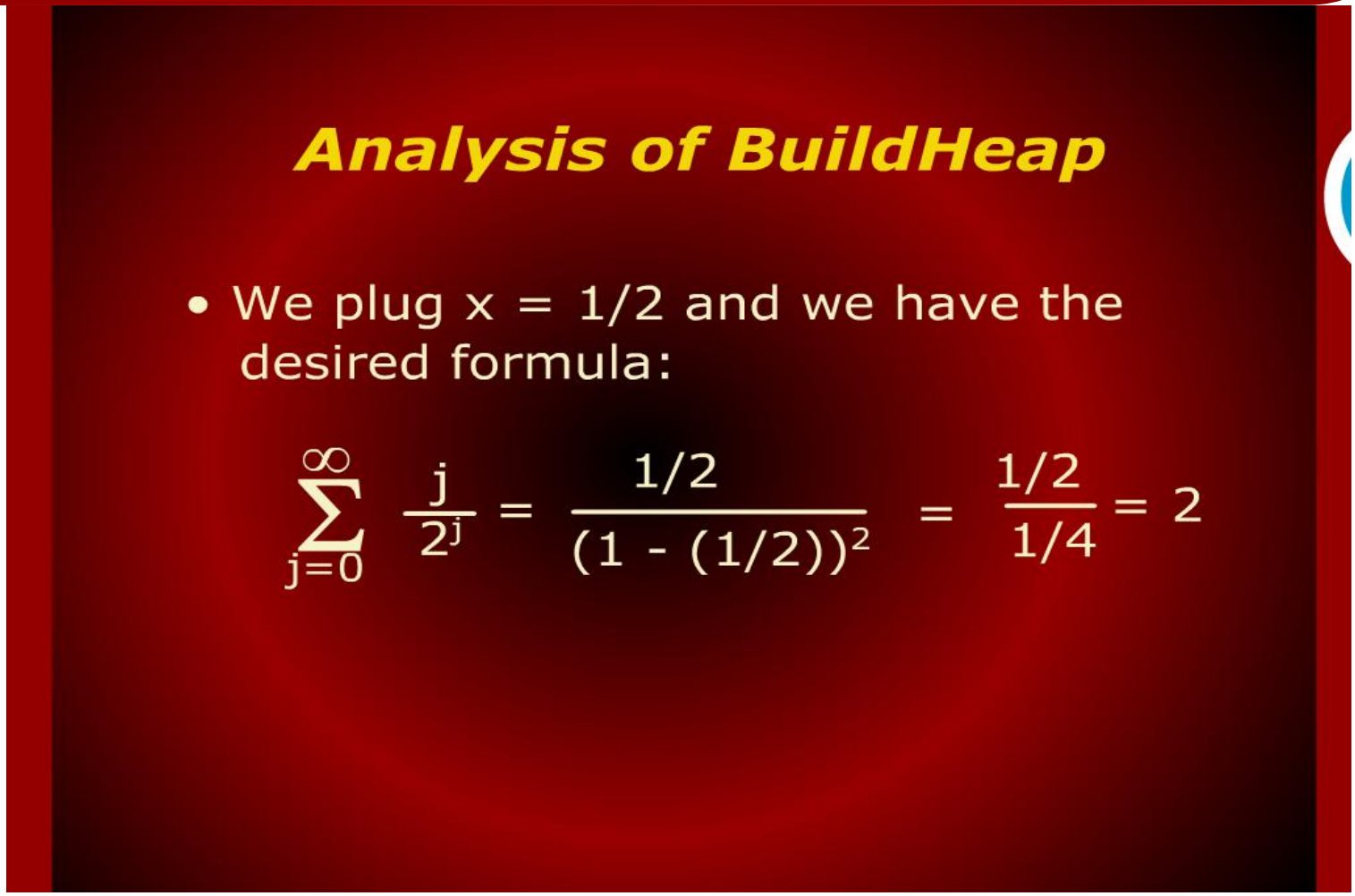
- How do we solve this sum? Recall the geometric series, for any constant $x < 1$

$$\sum_{j=0}^{\infty} x^j = \frac{1}{1-x}$$

- Take the derivative with respect to x and multiply by x

$$\sum_{j=0}^{\infty} jx^{j-1} = \frac{1}{(1-x)^2} \qquad \sum_{j=0}^{\infty} jx^j = \frac{x}{(1-x)^2}$$

Analysis of BuildHeap



Analysis of BuildHeap

Analysis of BuildHeap

- In our case, we have a bounded sum, but since the infinite series is bounded, we can use it as an easy approximation:

$$T(n) = 2^h \sum_{j=0}^{h} \frac{j}{2^j}$$

$$\leq 2^h \sum_{j=0}^{\infty} \frac{j}{2^j}$$

$$\leq 2^h \cdot 2 = 2^{h+1}$$

Analysis of BuildHeap

Analysis of BuildHeap

- Recall that $n = 2^{h+1} - 1$.
- Therefore

$$T(n) \leq n + 1 \in O(n)$$

- The algorithm takes at least $\Omega(n)$ time since it must access every element at once.
- So the total time for BuildHeap is $\Theta(n)$.

Analysis of BuildHeap

Analysis of BuildHeap

- BuildHeap is a relatively complex algorithm.
- Yet, the analysis yield that it takes $\Theta(n)$ time.
- An intuitive way to describe why it is so is to observe an important fact about binary trees
- The fact is that the vast majority of the nodes are at the lowest level of the tree.

Analysis of BuildHeap

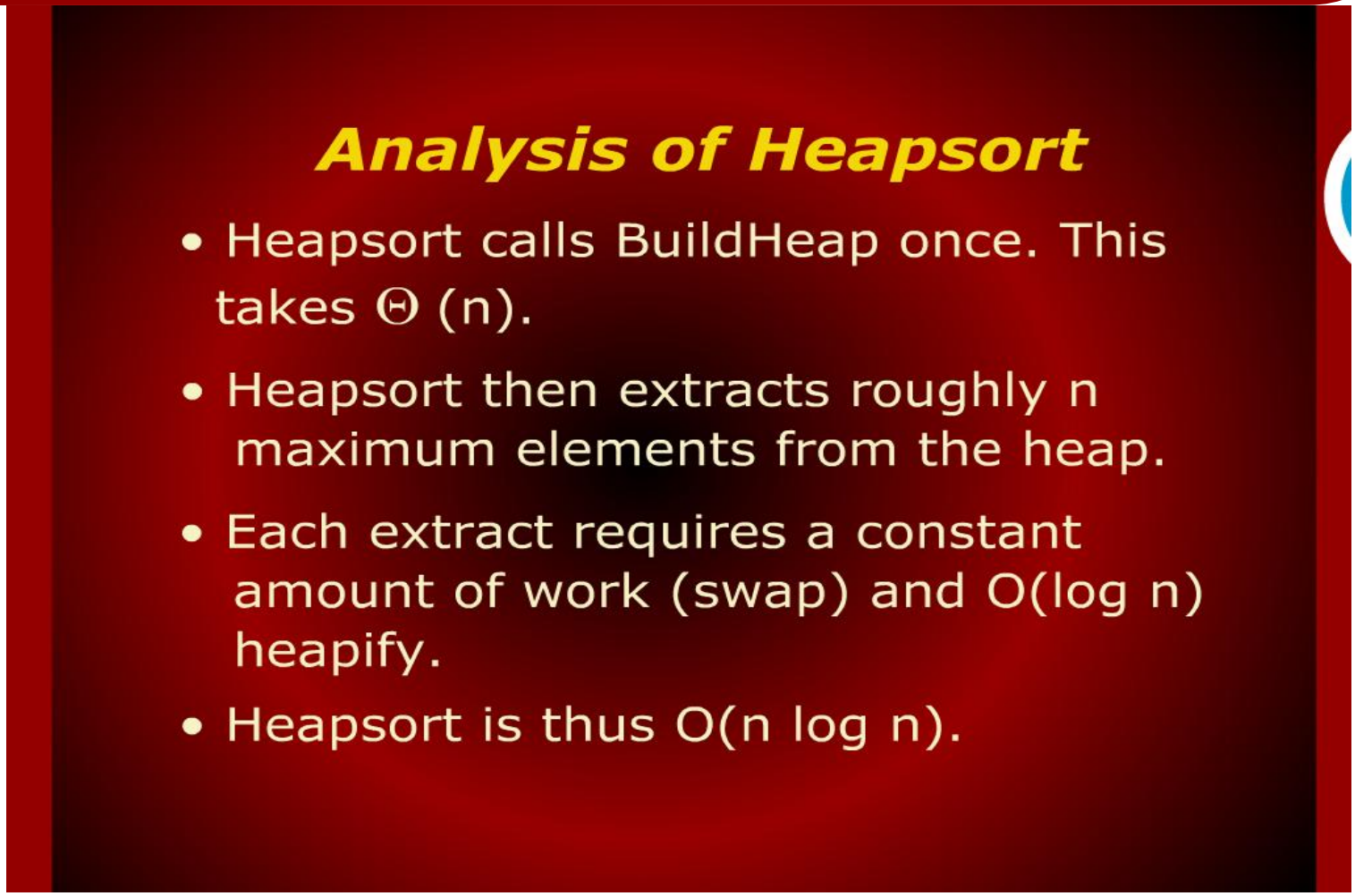
Analysis of BuildHeap

- The number of nodes at the bottom three levels alone is

$$2^h + 2^{h-1} + 2^{h-2} = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} = \frac{7n}{8} = 0.875n$$

- Almost 90% of the nodes of a complete binary tree reside in the 3 lowest levels.
- Thus, algorithms that operate on trees should be efficient (as BuildHeap is) on the bottom-most levels since that is where most of the weight of the tree resides.

Analysis of Heapsort



Analysis of Heapsort

Analysis of Heapsort

- Is HeapSort $\Theta(n \log n)$?
- The answer is yes.
- In fact, later we will show that comparison based sorting algorithms can not run faster than $\Omega(n \log n)$.
- Heapsort is such an algorithm and so is Mergesort that we saw ealier.